AUDITRA

Brand governance and content review

Evidence-linked findings, targeted corrections, accountable human approval.

Techcircuit Round 2 — The Build

Team Quoding

Arya Kashyap (Team Leader) · Ronit Mongia · Moli Verma

All screenshots are captured from the running application using the fictional Northstar Cloud demo workspace.

EXECUTIVE SUMMARY

Check content against the brand before it ships

Marketing and brand teams publish a constant stream of copy, and the review step is usually a manual read against a PDF of guidelines. Mistakes such as absolute guarantees, unapproved claims, missing disclosures and off-brand terminology are expensive to catch after publication.

AUDITRA is a working web application that checks copy against a brand's configured rulebook, shows each finding on the exact passage with the rule or source behind it, offers a targeted correction, and moves the piece through a human approval workflow with version history and an audit trail.

> **What is built in Round 2:** a full-stack app (React + FastAPI + SQLite) with a brand rulebook, a deterministic rule engine, an optional and grounded AI step with safe fallback, a review studio with inline highlights and one-click corrections, a server-enforced approval workflow, an approval queue, an audit trail, an overview dashboard and governance insights, all backed by persisted data.

Problem and target users

- **Content creators** who need to know, before submitting, what breaks brand policy and how to fix it.
- **Brand and compliance reviewers** who need to see what was flagged, on which version, and record an accountable decision.
- **Brand owners** who need to maintain the rulebook and see which rules are triggered most often.

Differentiators

- **Evidence first:** every rule finding carries exact character offsets and the configured source reference (for example, *Northstar Legal Governance Standard §4.1*). When no source exists, the UI says so instead of inventing one.
- **Deterministic core, optional AI:** rule checks work with no AI provider at all; AI findings are labelled as AI-assisted, unknown rule IDs are discarded, and provider failures fall back to rule findings.
- **Humans decide:** automated checks never approve content. The server rejects invalid transitions, and requesting changes or rejecting requires a written note.

IMPLEMENTED FEATURES

What works today

Area	Implemented behaviour
Workspaces	Multiple brand workspaces; seeded Northstar Cloud demo with 7 rules, 3 approved claims and 3 sample pieces; scoped demo reset.
Brand rulebook	Create, edit, toggle and delete rules in six categories with severity, match pattern, replacement and source reference; approved-claims registry; .txt/.md guideline import that turns matching lines into rules (created active, so they should be reviewed).
Rule engine	Prohibited wording, preferred terminology, missing required disclosures, restricted and unverified claims, style checks; exact offsets per finding.
AI step (optional)	Gemini, OpenAI or Anthropic when a key is configured; grounded against active rules; on failure the review completes with rule findings and records the failure.
Review studio	Editor and marked-up view, severity highlights, findings panel with counts and source filter, expandable findings, apply / edit-then-apply / dismiss, stale-review warnings, version history, discussion.
Corrections and versions	Applying a suggestion replaces only the matched passage and creates a new immutable version; re-review checks the new version.
Approval workflow	Draft → In review → Approved / Changes requested / Rejected, enforced by the server; warning before approving with open critical or high findings.
Approval queue	Status counts, queue items with highest severity and open findings, reviewer pane with copy, findings, discussion and decisions.
Audit trail	Chronological timeline of server-recorded events with actor, time, related content and labelled details.
Overview and insights	KPIs, items needing attention, workflow status, open findings by severity, findings by category, most-triggered rules, decision turnaround.
Demo roles	Switch between creator, reviewer and admin personas. This is a demo convenience, not authentication.

CORE WORKFLOW

From draft to accountable decision

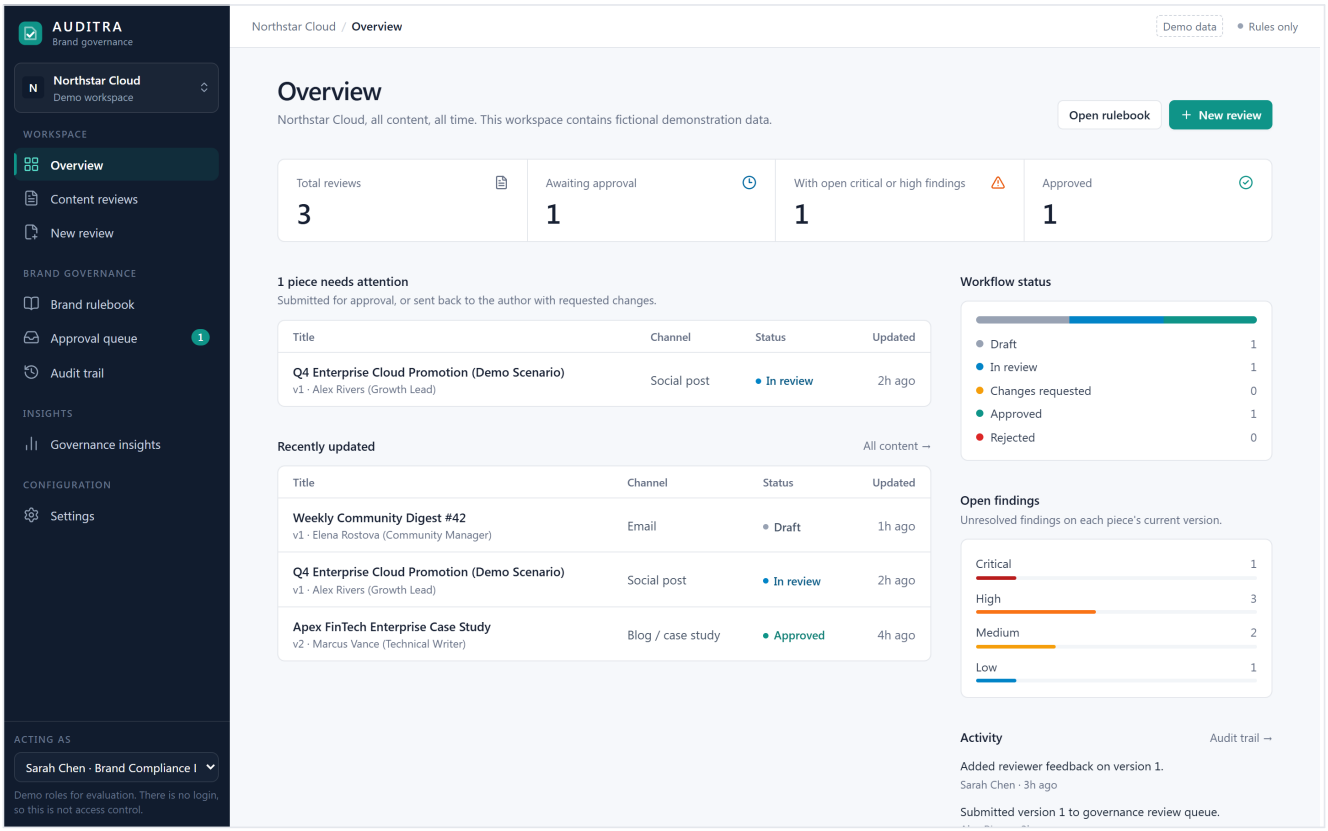


Figure 1. Overview: KPIs, items needing attention, workflow status and open findings by severity, all computed from stored data.

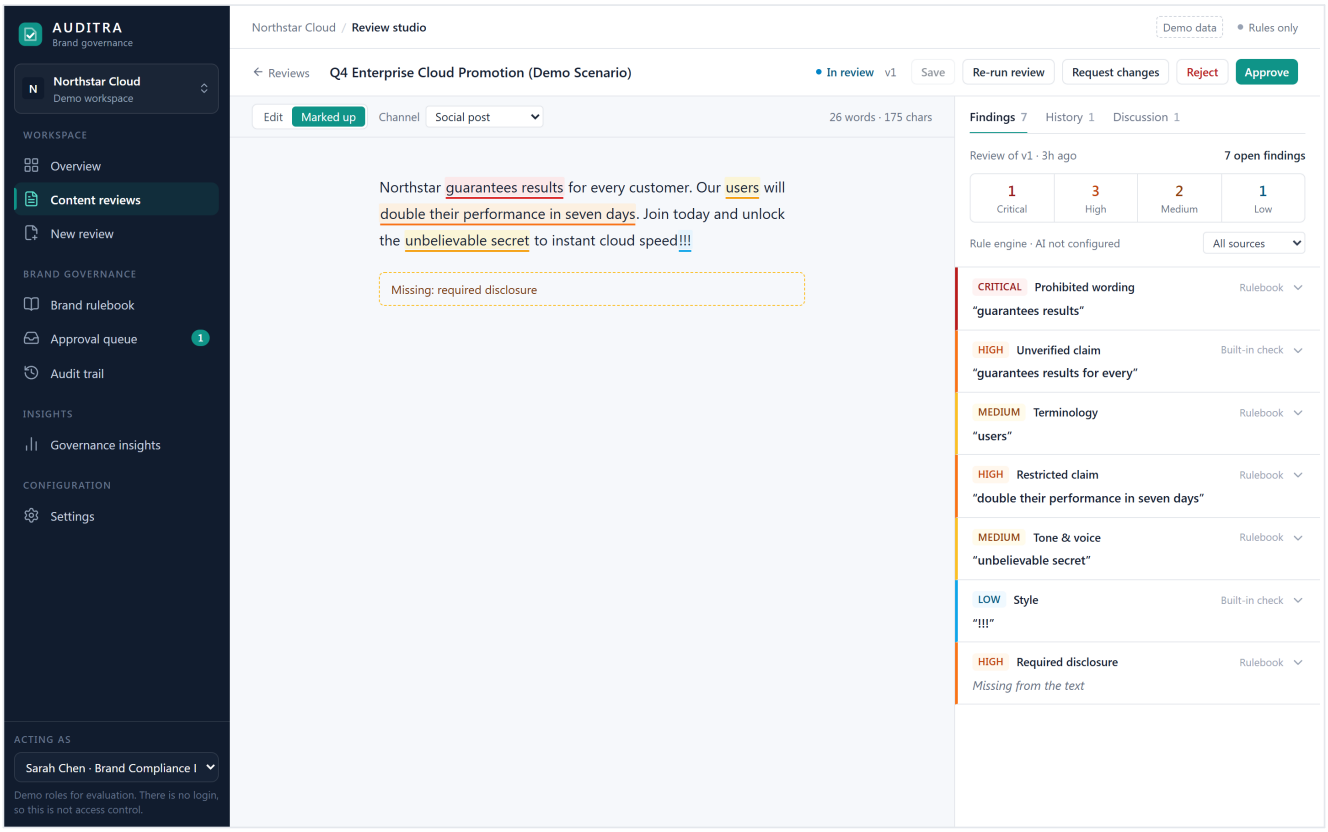


Figure 2. Review studio: marked-up copy with severity-coded highlights and the findings panel for the current version.

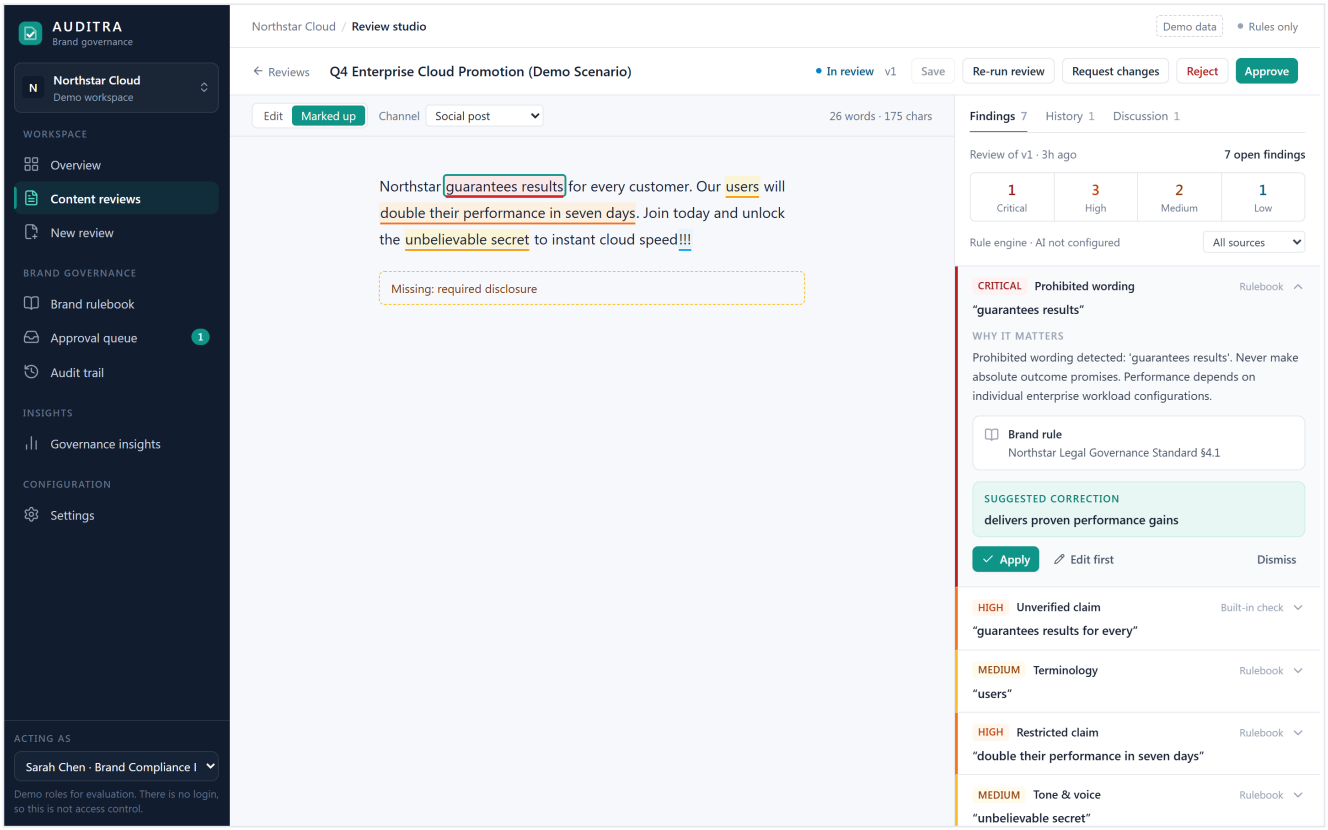


Figure 3. An expanded finding: why it matters, the brand rule it comes from, and a suggested correction with Apply, Edit first and Dismiss.

Steps a judge can follow

- Start a new review, load the labelled demo copy and run the review.
- Expand a finding to see the passage, the rule and the suggested correction; click it to jump to the passage.
- Apply the correction (a new version is saved), then re-run the review on the new version.
- Submit for approval, switch to the reviewer persona, and approve or request changes with a note.
- Open the audit trail to trace every step; refresh to confirm persistence.

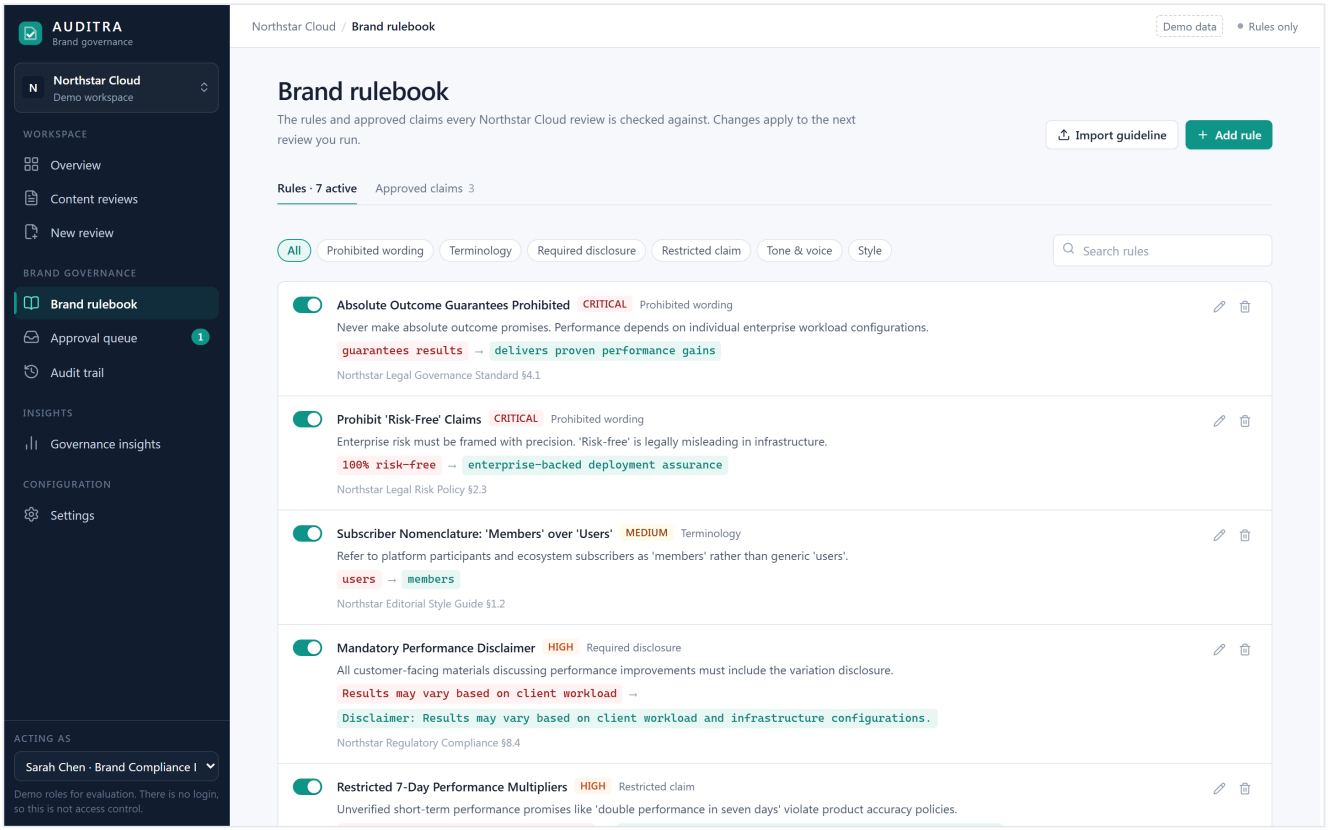


Figure 4. Brand rulebook: rules with category, severity, match pattern, suggested replacement and source reference.

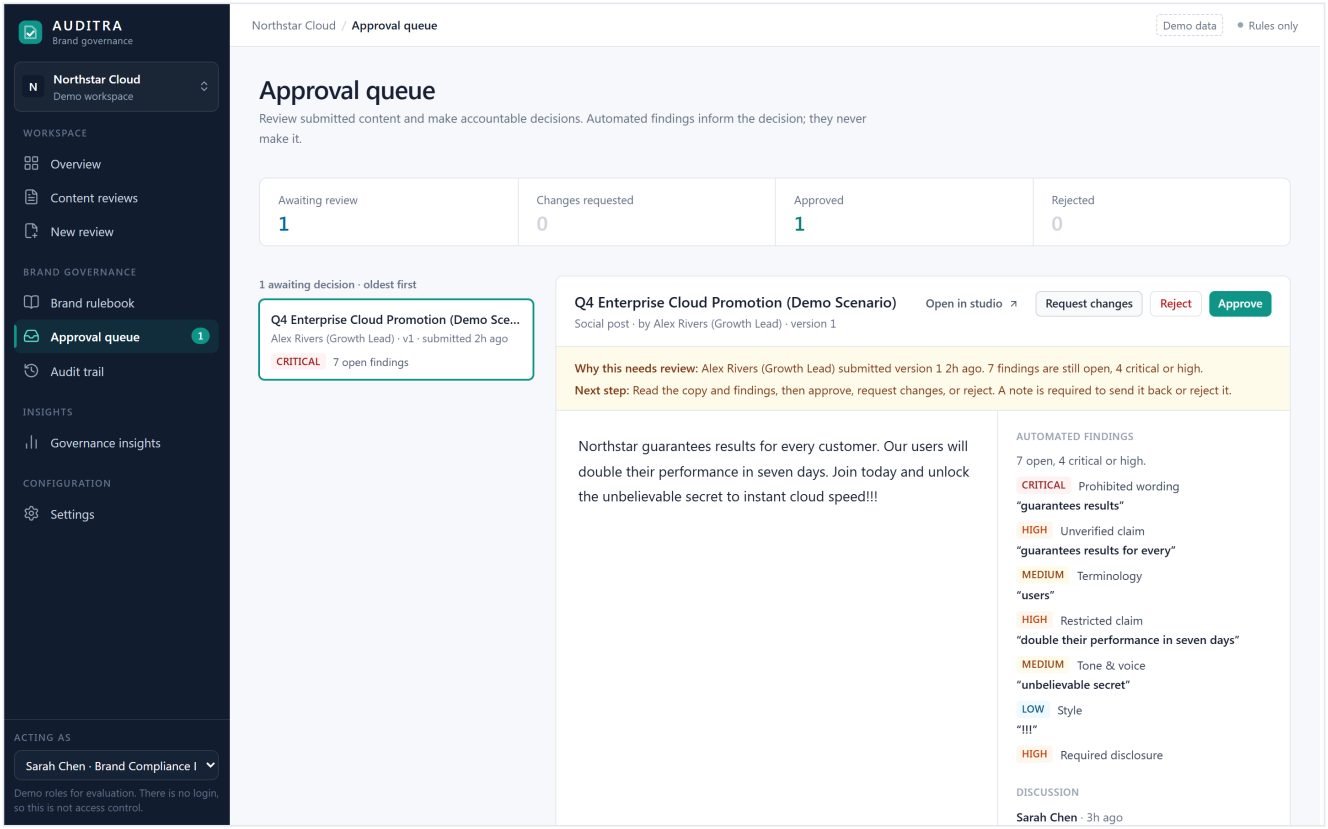


Figure 5. Approval queue: status counts, the waiting item with its highest severity, and a reviewer pane explaining why it needs review and what to do next.

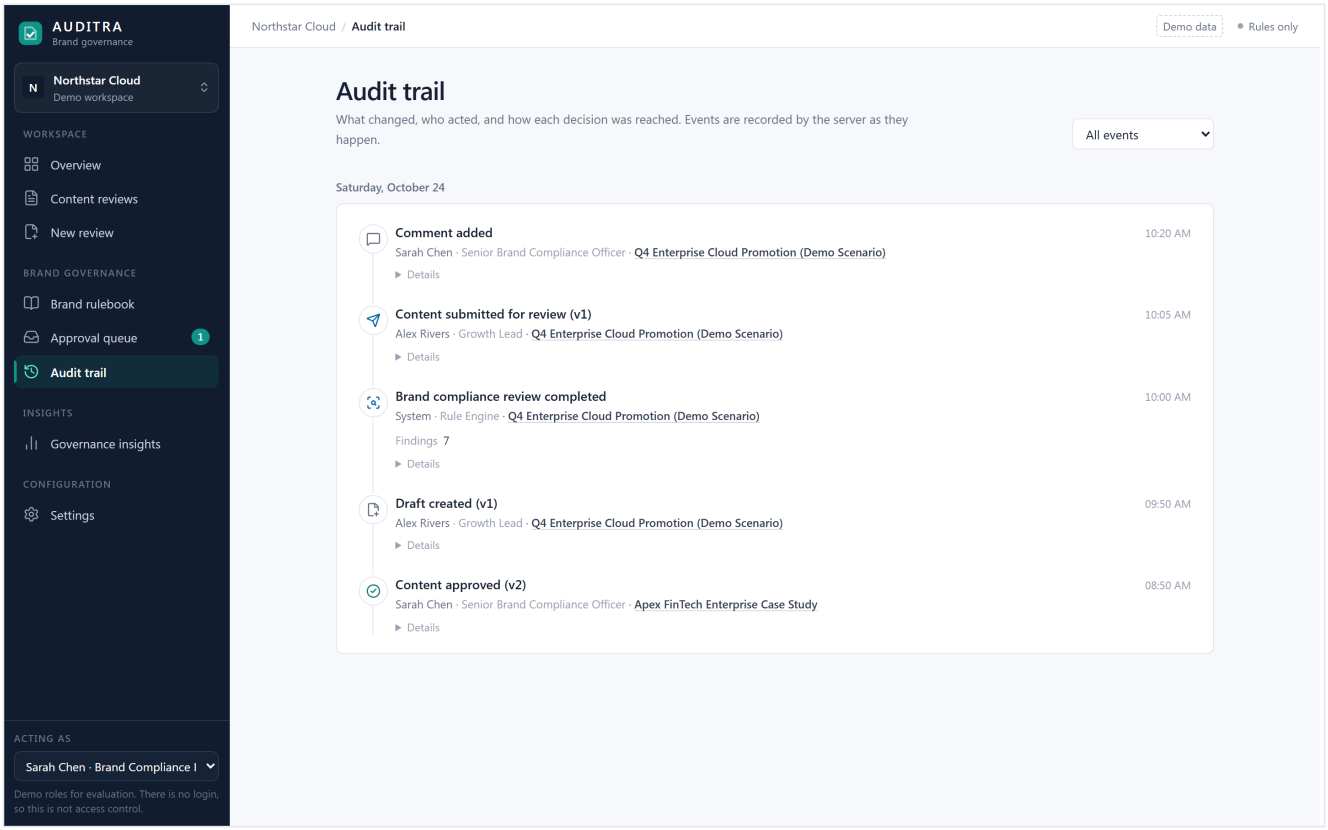


Figure 6. Audit trail: human-readable event timeline with actor, related content, version and labelled details.

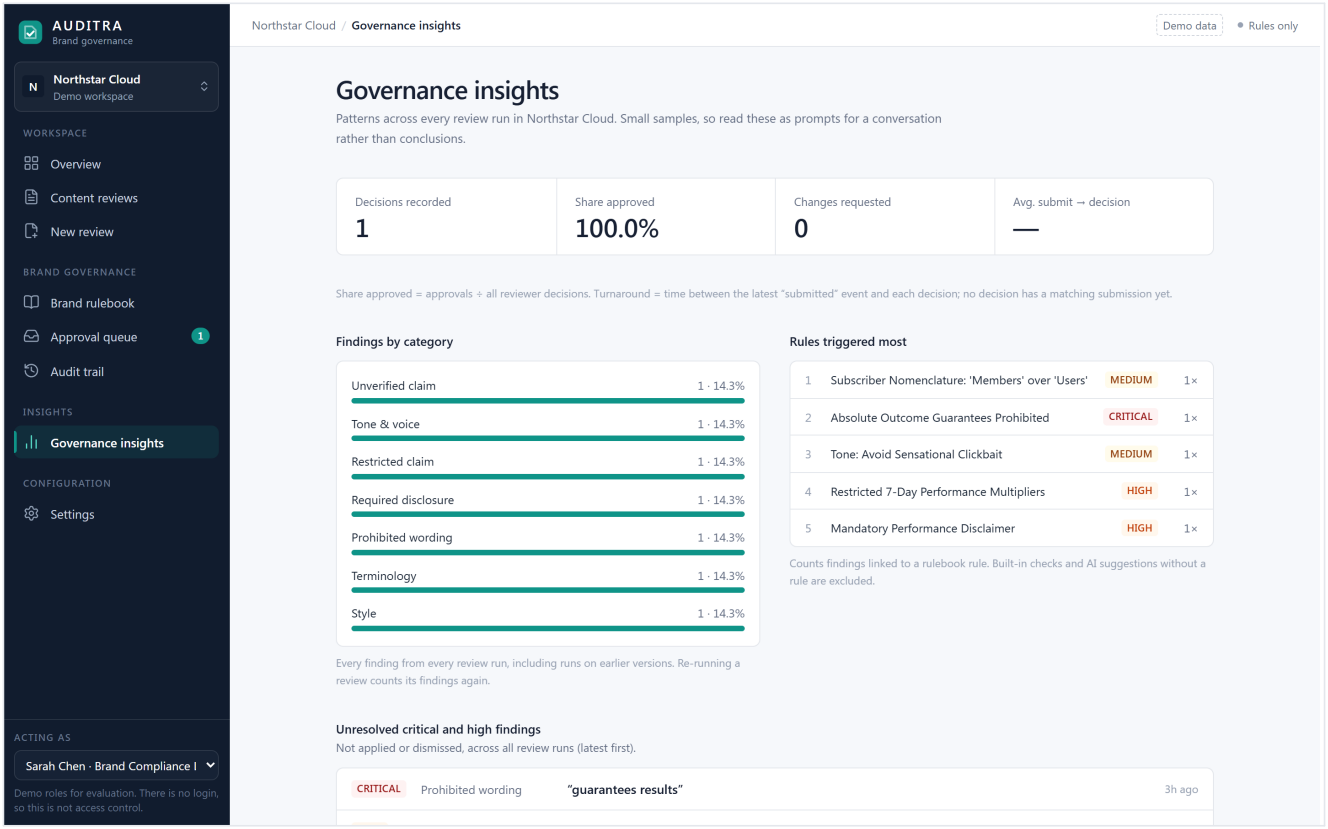


Figure 7. Governance insights: findings by category and most-triggered rules, with an explanation of how each figure is calculated.

TECHNICAL ARCHITECTURE

One process, one database, clear boundaries

Layer	Implementation
Frontend	React 19, TypeScript, Vite, Tailwind CSS, lucide-react icons. Hash-based routes so every screen survives a refresh. Shared component kit for buttons, badges, tables, dialogs and toasts.
API	FastAPI with Pydantic v2 schemas. Routes for brands, rules and claims, content, reviews, workflow, audit, dashboard and insights. Also serves the built frontend, so the app runs on one port.
Rule engine	Deterministic matcher producing findings with category, severity, offsets, explanation, source reference and suggested correction.
AI service	Optional provider call with the brand's active rules and claims in the prompt; output validated against stored rule IDs and the actual text.
Persistence	SQLAlchemy 2 over SQLite in WAL mode with foreign keys: brands, rules, claims, content items, immutable versions, review runs, findings, comments, decisions, audit events.

Data flow for a review

The studio saves the text as a content version. `POST /api/content/{id}/review` runs the rule engine (plus the AI step if configured) against the brand's active rules and claims and stores the findings against that version. Applying a finding creates the next version. Submit and decision endpoints move the item through the state machine, and every step writes an audit event.

AI analysis and evidence grounding

- With no provider key, the health endpoint reports *deterministic_fallback*, the UI shows **Rules only**, and reviews use the rule engine alone.
- When a provider is configured, AI findings whose rule ID does not exist are stripped of it, offsets are re-located in the real text, and findings are marked as AI-assisted with a human-verification flag.
- If the provider call fails, the review still completes with rule findings and the run records *ai_status: failed*. This path was tested with an invalid key.

Human approval, history and auditability

- Content versions are immutable; each correction or edit creates a new version, and findings are tied to the version they were raised on. The studio warns when findings belong to an older version.
- Decisions record the reviewer, decision, note and version. The server refuses invalid transitions (for example, approving a draft).
- The audit trail is an append-only application log in the database. It is not presented as tamper-proof.

TESTING AND VALIDATION

What was actually verified

Check	Result (24 Oct 2026)
Backend test suite (pytest, isolated temporary database)	11 passed
Frontend production build (tsc -b and vite build)	Succeeds
Ad-hoc HTTP smoke script (not part of the repository) on a running server with a throwaway database	21 of 21 checks passed: rule create, edit and validation; content creation; rule findings with offsets and source; deterministic fallback; correction to v2; re-review; state-machine enforcement; dashboard and queue consistency; comments; approval; version history; audit lifecycle; 404 handling
Persistence across a server restart	Brand, approved status and audit trail all present after restart
AI provider failure	Review completes with rule findings and ai_status: failed
Browser check (Chrome via Playwright)	All screens at 1440×900, 1024×768, 768×1024 and 375×812: no console errors, no failed API calls, no horizontal overflow; full creator-to-reviewer journey completed in the UI
Successful live AI call	**Not tested**: no provider key was available

Defects found and fixed during validation included: new reviews failing to save (a schema required a field the frontend did not send), test runs writing into the demo database, an overview panel contradicting the approval count, raw validation JSON in error messages, and the frontend dev server lacking an API proxy.

Round 1 proposal compared with the Round 2 build

Compared item by item with our Round 1 submission (*Round 1: The Plan*). Status reflects the code as submitted.

Round 1 plan	Round 2 status
Brand rulebook: guidelines, tone, facts, claims, terminology	**Built.** Rules in six categories plus an approved-claims registry. Logo and visual assets are not supported.
Text submission, rule checks, explainable findings, suggestions	**Built.** Deterministic checks with exact offsets, explanations, source references and targeted corrections.
AI-assisted analysis with validated, labelled output	**Built, not demonstrated live.** Grounding and the fallback are implemented; no successful live provider call was tested.
Review workflow: comments, request changes, approve, reject	**Built**, enforced by the server.
Revision history and audit trail	**Built.** Immutable versions and a server-written event log.
Governance insights	**Built.** Category, rule and turnaround summaries from stored data.
Role-based permissions (privacy and security section)	**Not built.** There is a demo role switcher only; there is no login or access control.
Document upload and text extraction	**Partial.** .txt and .md guideline import only; no PDF, DOCX or content-document upload.

Round 1 plan	Round 2 status
OCR and visual checks (could-have)	**Not built.** Moved to Round 3.
Database: PostgreSQL (SQLite for prototype)	**SQLite**, configurable through DATABASE_URL; not tested on PostgreSQL.
Multiple brand workspaces (could-have)	**Built.**
Testing with sample brands; user feedback	**Partial.** Automated tests and a browser walkthrough on a fictional brand; no user interviews and no false-positive measurement yet.
Deployed prototype and judge-accessible repository	**Pending.** Not deployed and not yet pushed at the time of writing.

CURRENT LIMITATIONS

What is not done yet

- **No authentication or authorisation.** The role switcher is for demonstration only.
- **Deployment:** no public deployment has been verified at the time of writing.
- The rule engine is pattern-based and does not understand paraphrase; the AI step can help but needs human verification.
- Text content only: no images, video or rich-text formatting.
- Guideline import accepts UTF-8 text and Markdown only (no PDF or DOCX extraction), and imported rules are active immediately, so an imported document takes effect before anyone reviews it.
- No structured evaluation set yet: we have not measured detection rate, false positives or reviewer agreement, and no user interviews have been run.
- SQLite suits a single node; concurrent multi-user production use needs a server database.
- The audit log is an application table, not a tamper-proof ledger.

ROUND 3

Three improvements and how we will measure them

1. Multimodal brand governance for visual creatives

Value. Extend the same rulebook to banners and social images: palette and logo usage, minimum contrast, and OCR of on-image text checked by the existing rule engine.

Evaluation. Precision and recall on a labelled set of compliant and non-compliant creatives; share of on-image text violations caught that text-only review misses; reviewer time per creative.

2. Evidence-grounded brand knowledge and claim verification

Value. Retrieve from approved source documents (product facts, certifications, policy) so each claim is matched to a dated, attributable passage, with expiry for stale evidence.

Evaluation. Share of flagged claims linked to a correct source passage; false 'verified' rate (target near zero); reviewer agreement with the suggested evidence.

3. Enterprise governance and publishing integrations

Value. Real identity (SSO) and roles, multi-step approval policies for high-risk content, and connectors so only approved versions are published to CMS and scheduling tools.

Evaluation. Zero unapproved versions published in integration tests; median time from submission to decision; policy coverage (share of content routed by a defined policy).

IMPLEMENTATION DETAILS

Technology stack and links

Component	Technology
Frontend	React 19, TypeScript 6, Vite 8, Tailwind CSS 3.4, lucide-react
Backend	Python 3.12, FastAPI, Pydantic v2, Uvicorn, SQLAlchemy 2
Database	SQLite with WAL and foreign keys (configurable via DATABASE_URL)
AI providers (optional)	Gemini, OpenAI or Anthropic via server-side environment variables
Testing	pytest with FastAPI TestClient (in tests/); browser checks were run manually with Playwright and are not part of the repository
Run locally	pip install -r requirements.txt; npm install and npm run build in frontend/; uvicorn backend.main:app --port 8000

Project links

Live application: not yet deployed; no public URL has been verified.
Source repository: not yet published; no remote is configured for the local repository.
These links will be added once the team publishes the repository and deploys the app.